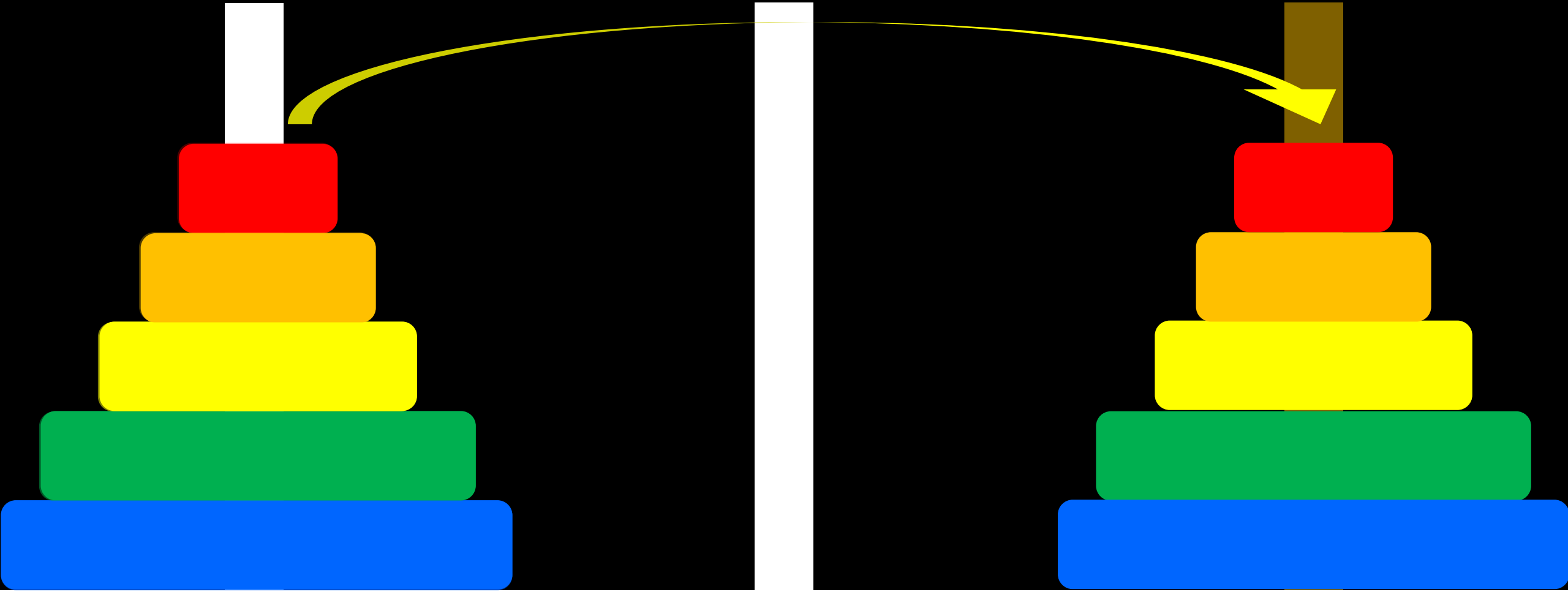


Chapter 18

Recursion

Motivations





Objectives

- 7.1** To describe what a recursive method is and the benefits of using recursion (§18.1).
- 7.2** To develop recursive methods for recursive mathematical functions (§§18.2–18.3).
- 7.3** To explain how recursive method calls are handled in a call stack (§§18.2–18.3).
- 7.4** To solve problems using recursion (§18.4).
- 7.5** To use an overloaded helper method to derive a recursive method (§18.5).
- 7.6** To implement a selection sort using recursion (§18.5.1).
- 7.7** To implement a binary search using recursion (§18.5.2).
- 7.8** To get the directory size using recursion (§18.6).
- 7.9** To solve the Tower of Hanoi problem using recursion (§18.7).
- 7.10** To draw fractals using recursion (§18.8).
- 7.11** To discover the relationship and difference between recursion and iteration (§18.9).
- 7.12** To know tail-recursive methods and why they are desirable (§18.10).

Objectives

- 7.1 To describe what a **recursive** method is and the benefits of using **recursion** (§18.1).
- 7.4 To solve problems using **recursion** (§18.4).
- 7.5 To use an **overloaded helper** method to derive a recursive method (§18.5).
- 7.6 To implement a selection **sort** using recursion (§18.5.1).
- 7.7 To implement a binary **search** using recursion (§18.5.2).
- 7.8 To get the **directory size** using recursion (§18.6).
- 7.9 To solve the **Tower of Hanoi** problem using recursion (§18.7).
- 7.10 To draw **fractals** using recursion (§18.8).
- 7.11 To discover the relationship and difference between **recursion** and **iteration** (§18.9).
- 7.12 To know **tail-recursive methods** and why they are desirable (§18.10).

Factorial

1

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$3! = 3 \times 2 \times 1 = 6$$

$$2! = 2 \times 1 = 2$$

$$1! = 1 = ?$$

?



Factorial

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0 \quad = n(n-1)!$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$3! = 3 \times 2 \times 1 = 6$$

$$2! = 2 \times 1 = 2$$

$$1! = 1 = 1$$

Factorial

1

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0 = n(n-1)!$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720 = 6(5)!$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120 = 5(4)!$$

$$4! = 4 \times 3 \times 2 \times 1 = 24 = 4(3)!$$

$$3! = 3 \times 2 \times 1 = 6 = 3(2)!$$

$$2! = 2 \times 1 = 2 = 2(1)!$$

$$1! = 1 = 1 = 1(0)!$$

$$0! = ? \text{ by definition}$$

Factorial

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0 = n(n-1)!$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720 = 6(5)!$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120 = 5(4)!$$

$$4! = 4 \times 3 \times 2 \times 1 = 24 = 4(3)!$$

$$3! = 3 \times 2 \times 1 = 6 = 3(2)!$$

$$2! = 2 \times 1 = 2 = 2(1)!$$

$$1! = 1 = 1 = 1(0)!$$

$$0! = 1 \quad \text{by definition}$$



Factorial

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0 = n(n-1)!$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720 = 6(5)!$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120 = 5(4)!$$

$$4! = 4 \times 3 \times 2 \times 1 = 24 = 4(3)!$$

$$3! = 3 \times 2 \times 1 = 6 = 3(2)!$$

$$2! = 2 \times 1 = 2 = 2(1)!$$

$$1! = 1 = 1 = 1$$

$$0! = 1$$



Factorial: loop

1

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

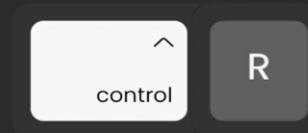
patterns: ?

result = result * i

Factorial: loop

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$



Factorial: recursion

$$n! = n(n-1)(n-2)\dots 2 \times 1 \quad n > 0$$


$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$4! = 4 \times 3 \times 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

$$2! = 2 \times 1$$

$$1! = 1$$

$$0! = 1$$

Factorial: recursion

$$n! = n(n-1)! \quad n > 0$$

$$6! = 6 \times 5!$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$4! = 4 \times 3 \times 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

$$2! = 2 \times 1$$

$$1! = 1$$

$$0! = 1$$

Factorial: recursion

$$n! = n(n-1)! \quad n > 0$$

$$6! = 6 \times 5!$$

$$5! = 5 \times 4!$$

$$4! = 4 \times 3 \times 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

$$2! = 2 \times 1$$

$$1! = 1$$

$$0! = 1$$

Factorial: recursion

$$n! = n(n-1)! \quad n > 0$$

$$6! = 6 \times 5!$$

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2 \times 1$$

$$2! = 2 \times 1$$

$$1! = 1$$

$$0! = 1$$

Factorial: recursion

$$n! = n(n-1)! \quad n > 0$$

$$6! = 6 \times 5!$$

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2!$$

$$2! = 2 \times 1$$

$$1! = 1$$

$$0! = 1$$

Factorial: recursion

$$n! = n(n-1)! \quad n > 0$$

$$6! = 6 \times 5!$$

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2!$$

$$2! = 2 \times 1!$$

$$1! = 1 \cdot 1$$

$$0! = 1 \quad \text{by definition}$$

Factorial: recursion

$$n! = n(n-1)! \quad n > 0$$

$$6! = 6 \times 5!$$

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2!$$

$$2! = 2 \times 1!$$

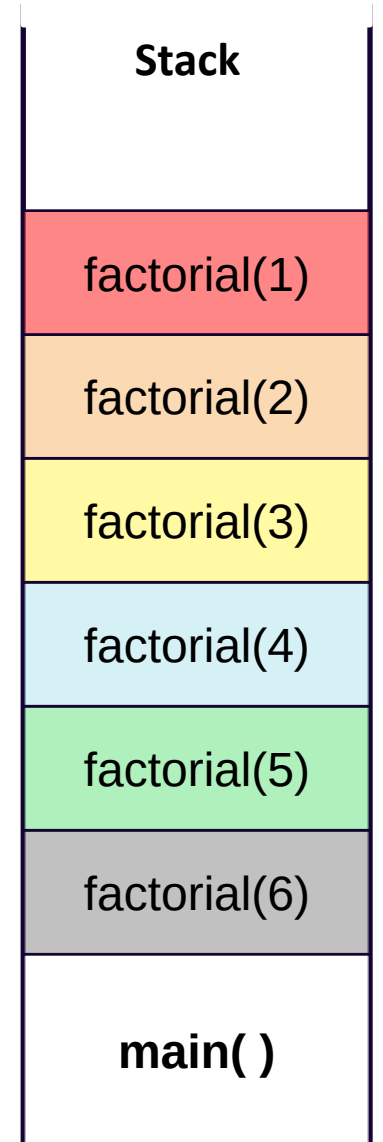
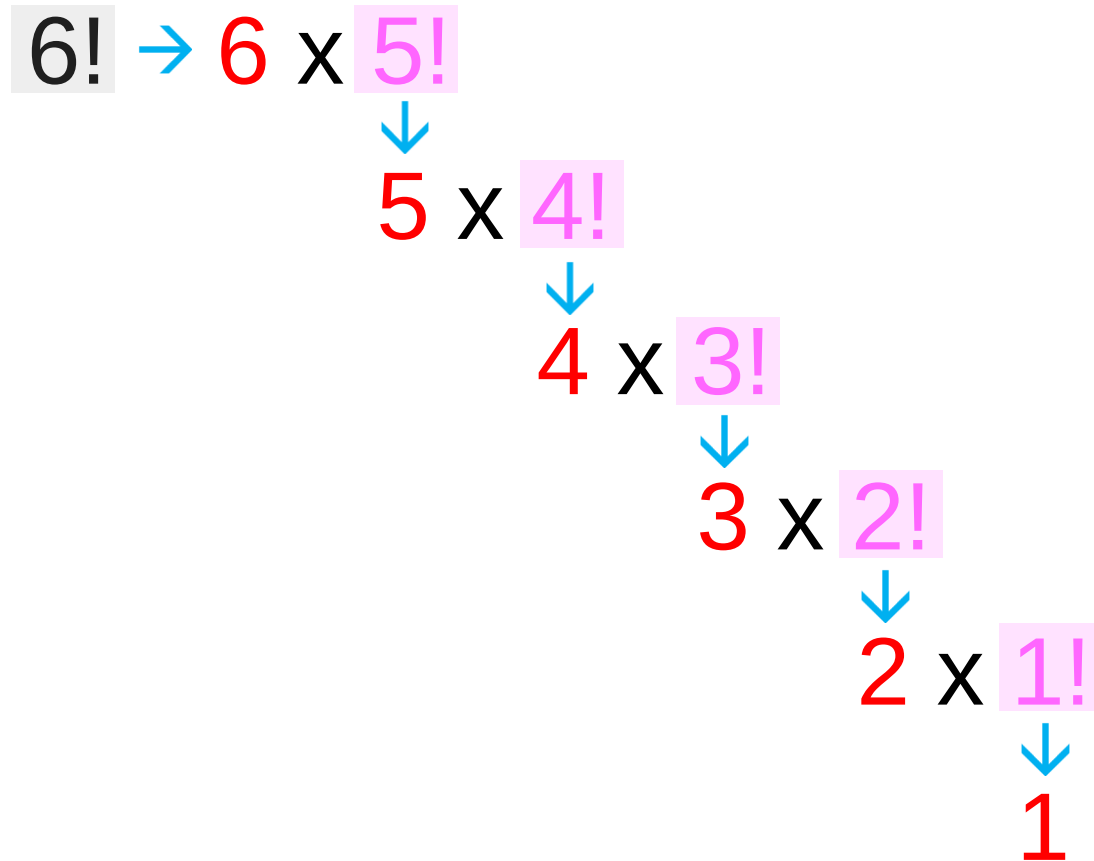
$$1! = 1$$

$$0! = 1$$

Factorial: recursion

$$n! = n(n-1)!$$

$$n > 0$$



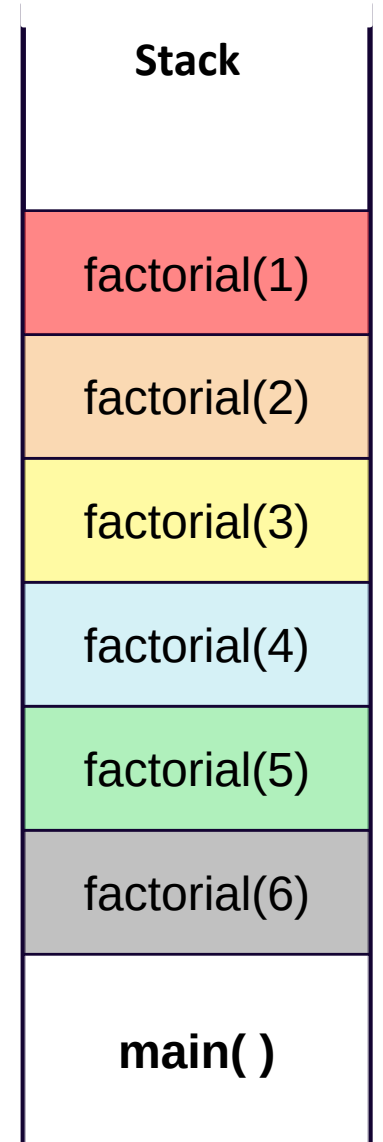
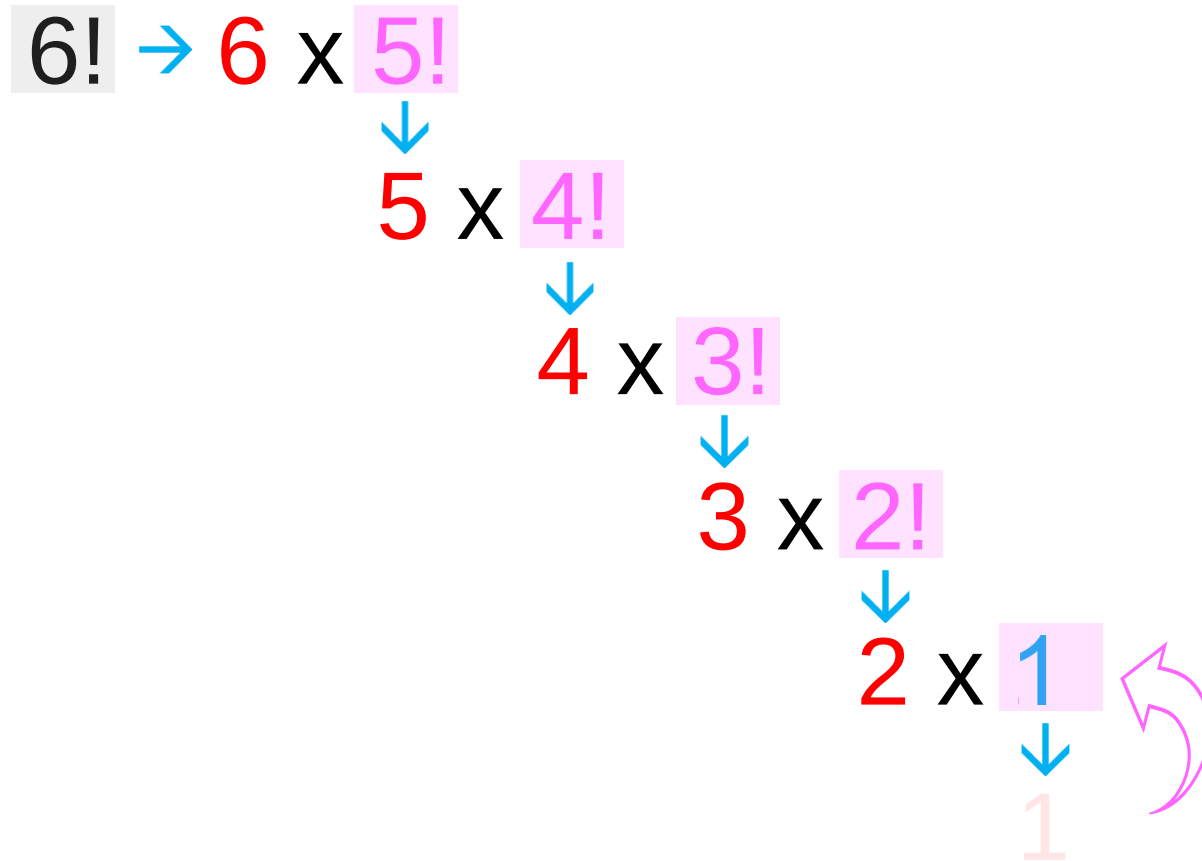
$$0! =$$

1

Factorial: recursion

$$n! = n(n-1)!$$

$$n > 0$$



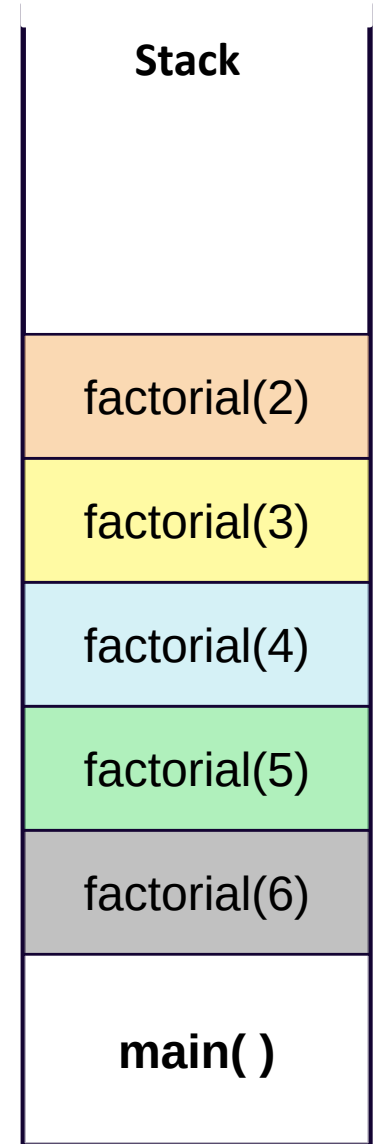
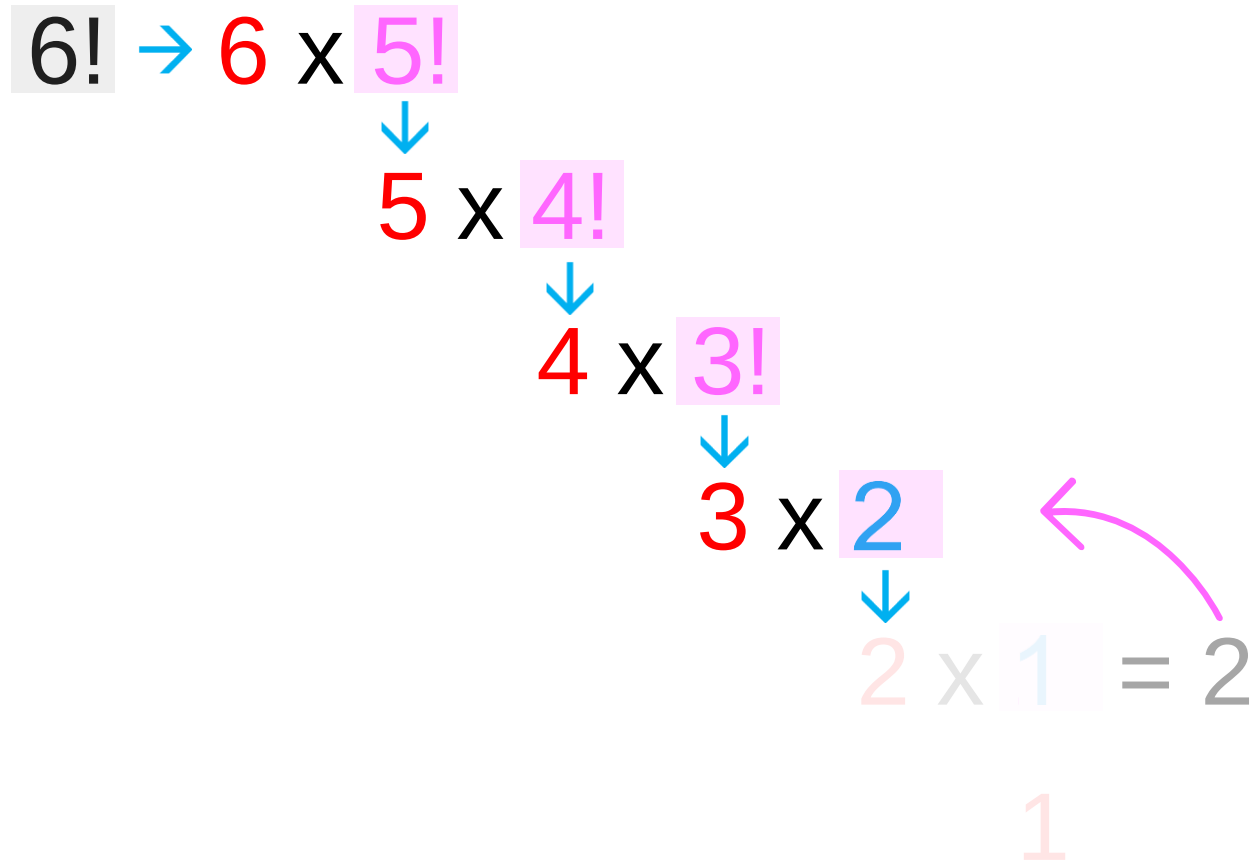
$$0! =$$

1

Factorial: recursion

$$n! = n(n-1)!$$

$$n > 0$$



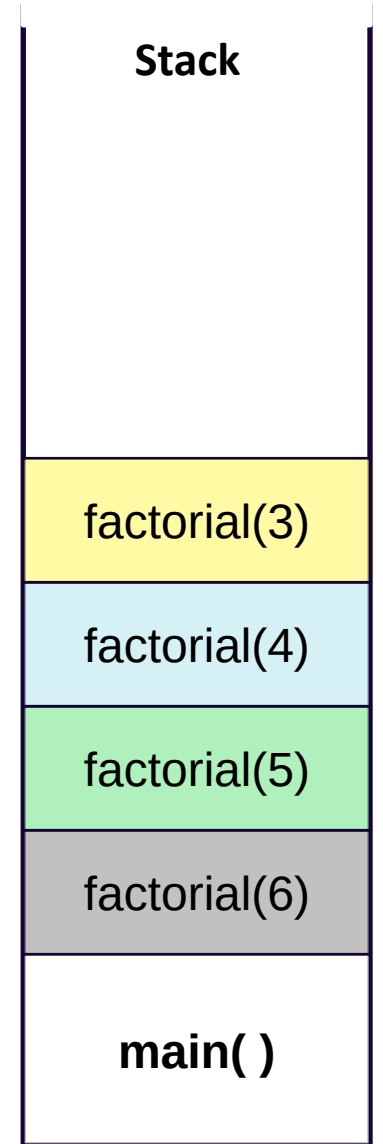
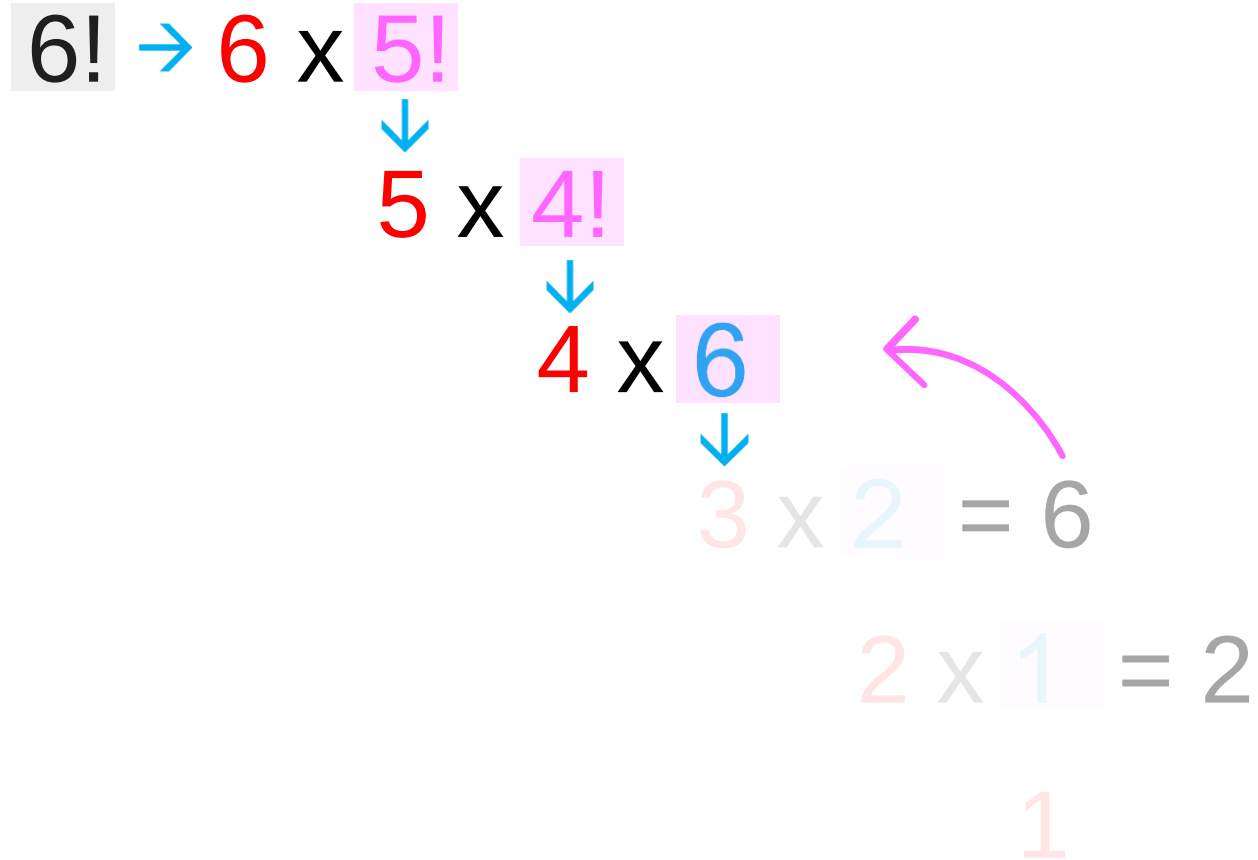
$$0! =$$

$$1$$

Factorial: recursion

$$n! = n(n-1)!$$

$$n > 0$$



$$0! =$$

$$1$$

Factorial: recursion

$$n! = n(n-1)!$$

$$n > 0$$

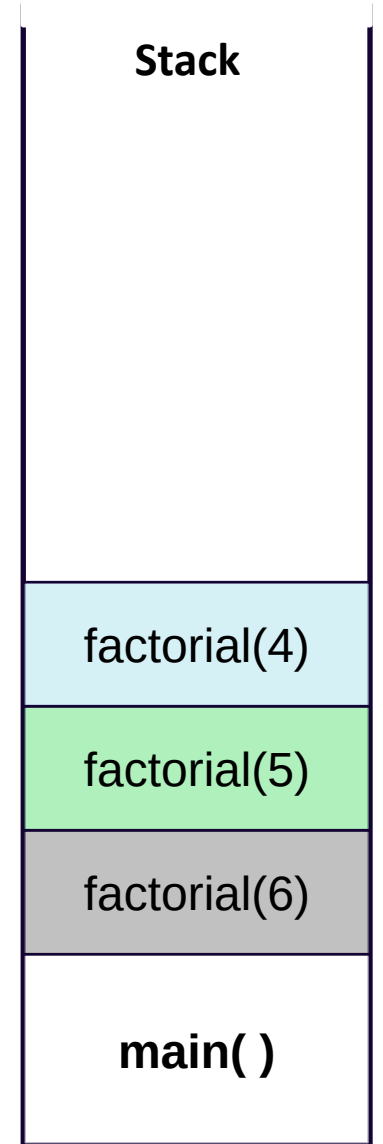
The diagram illustrates the recursive calculation of 6! through a series of steps, each with a colored box around the number being calculated:

- $6! \rightarrow 6 \times 5!$ (5! is in a pink box)
- $\downarrow$
- 5×24 (5 is in a red box, 24 is in a blue box)
- $\downarrow$
- $4 \times 6 = 24$ (4 is in a light orange box, 6 is in a light blue box)
- $\downarrow$
- $3 \times 2 = 6$ (3 is in a light orange box, 2 is in a light blue box)
- $\downarrow$
- $2 \times 1 = 2$ (2 is in a light orange box, 1 is in a light blue box)
- $\downarrow$
- 1 (in a light orange box)

A curved pink arrow points from the 24 in the third step to the 24 in the fourth step, indicating the return of the value from the recursive call.

$$0! =$$

$$1$$



Factorial: recursion

$$n! = n(n-1)!$$

$$n > 0$$

$$6! \rightarrow 6 \times 120$$



$$5 \times 24 = 120$$

$$4 \times 6 = 24$$

$$3 \times 2 = 6$$

$$2 \times 1 = 2$$

1

$$0! =$$

1

Stack

factorial(5)

factorial(6)

main()

Factorial: recursion

$$n! = n(n-1)!$$

$6!$ $6 \times 120 = 720$ 

$n > 0$

$$5 \times 24 = 120$$

$$4 \times 6 = 24$$

$$3 \times 2 = 6$$

$$2 \times 1 = 2$$

1

$$0! =$$

1

Stack

factorial(6)

main()

Recursion

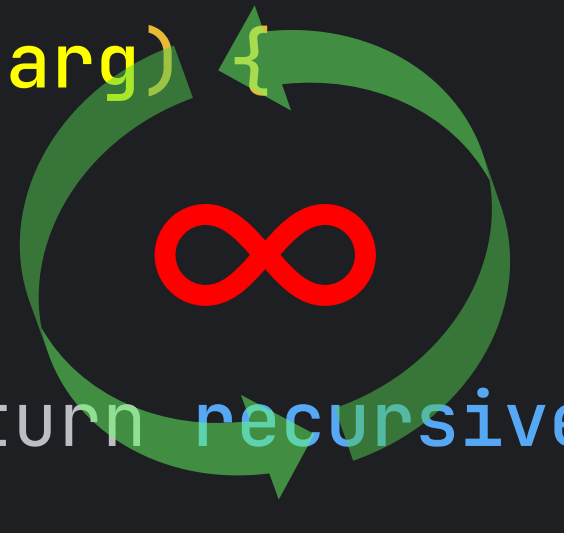
- borrowed from Late Latin *recursiōn-*, *recursiō* "return,"
- from Latin *recurrere* "to run back, return" + *-tiōn-*, *-tiō*, suffix of verbal action

Recursive

- of, relating to, or constituting a procedure that can repeat itself indefinitely

Recursion: a method calling itself

```
recursive(arg) {  
    return recursive(arg)  
}
```



A diagram illustrating the concept of recursion. It features a green circular arrow that starts at the opening curly brace of the function definition and loops back to the same point, symbolizing a self-call. In the center of this loop is a red infinity symbol (∞), representing the infinite nature of recursive calls. The code snippet shows a function named 'recursive' that takes an argument 'arg' and returns the result of 'recursive(arg)', which is the recursive call.

Recursion: a method calling itself

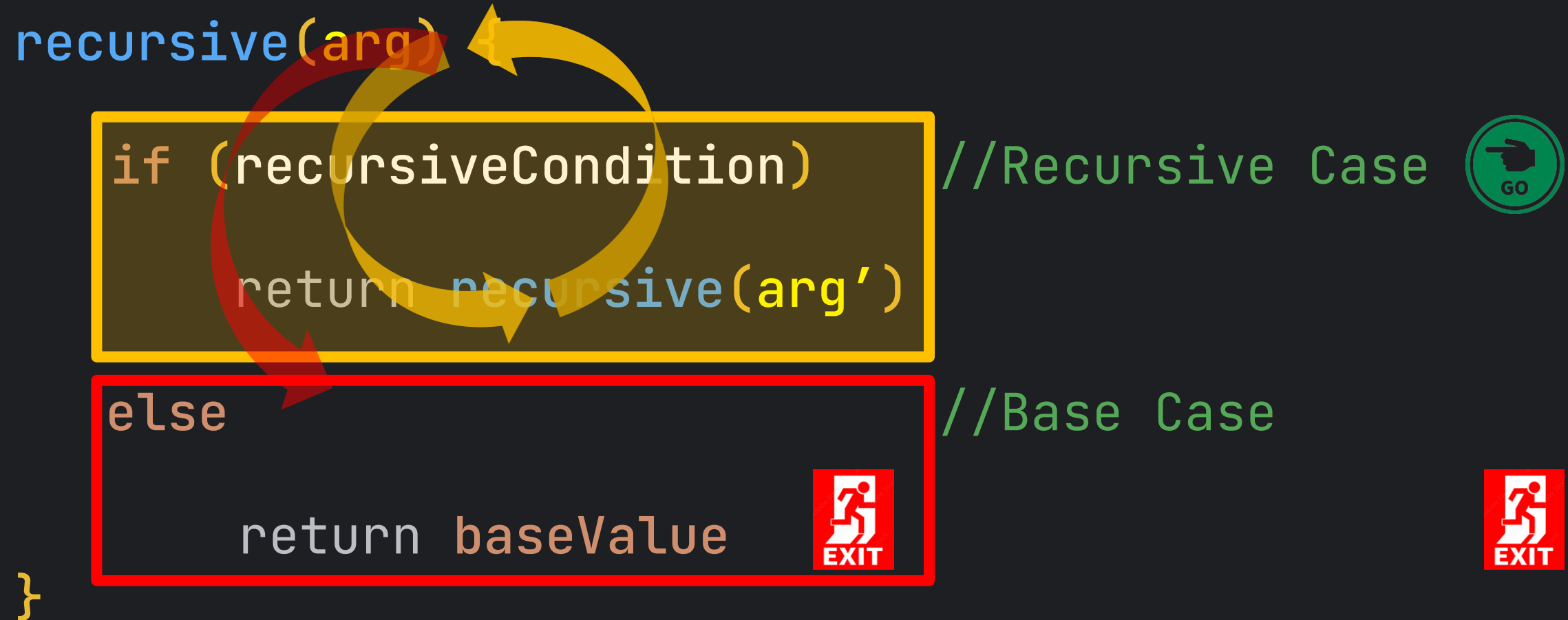
```
recursive(arg) {
```

```
    return recursive(arg')
```



```
}
```

Recursion: a method calling itself



Recursion: a method calling itself

```
recursive(arg) {
```

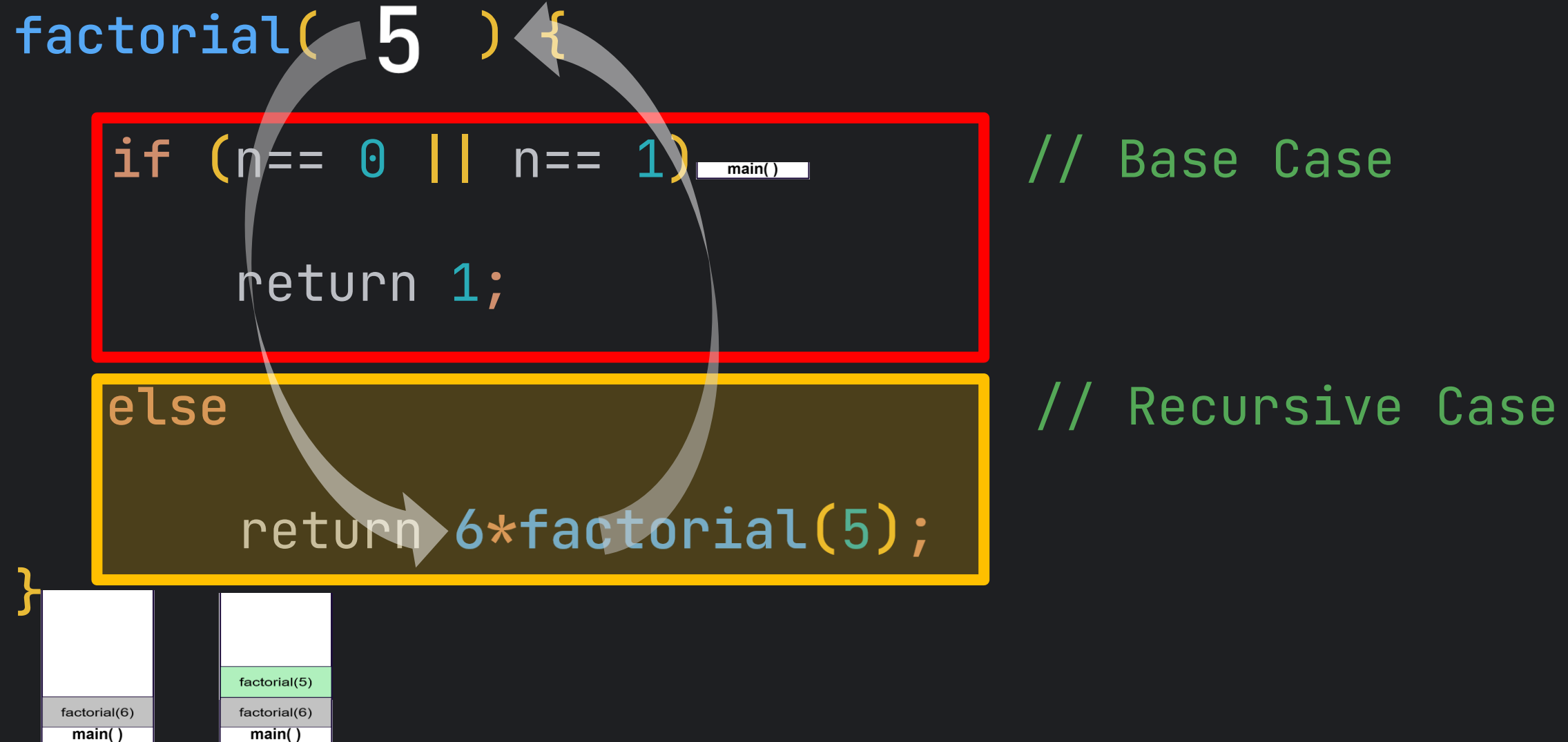
```
    return baseValue
```

```
}
```

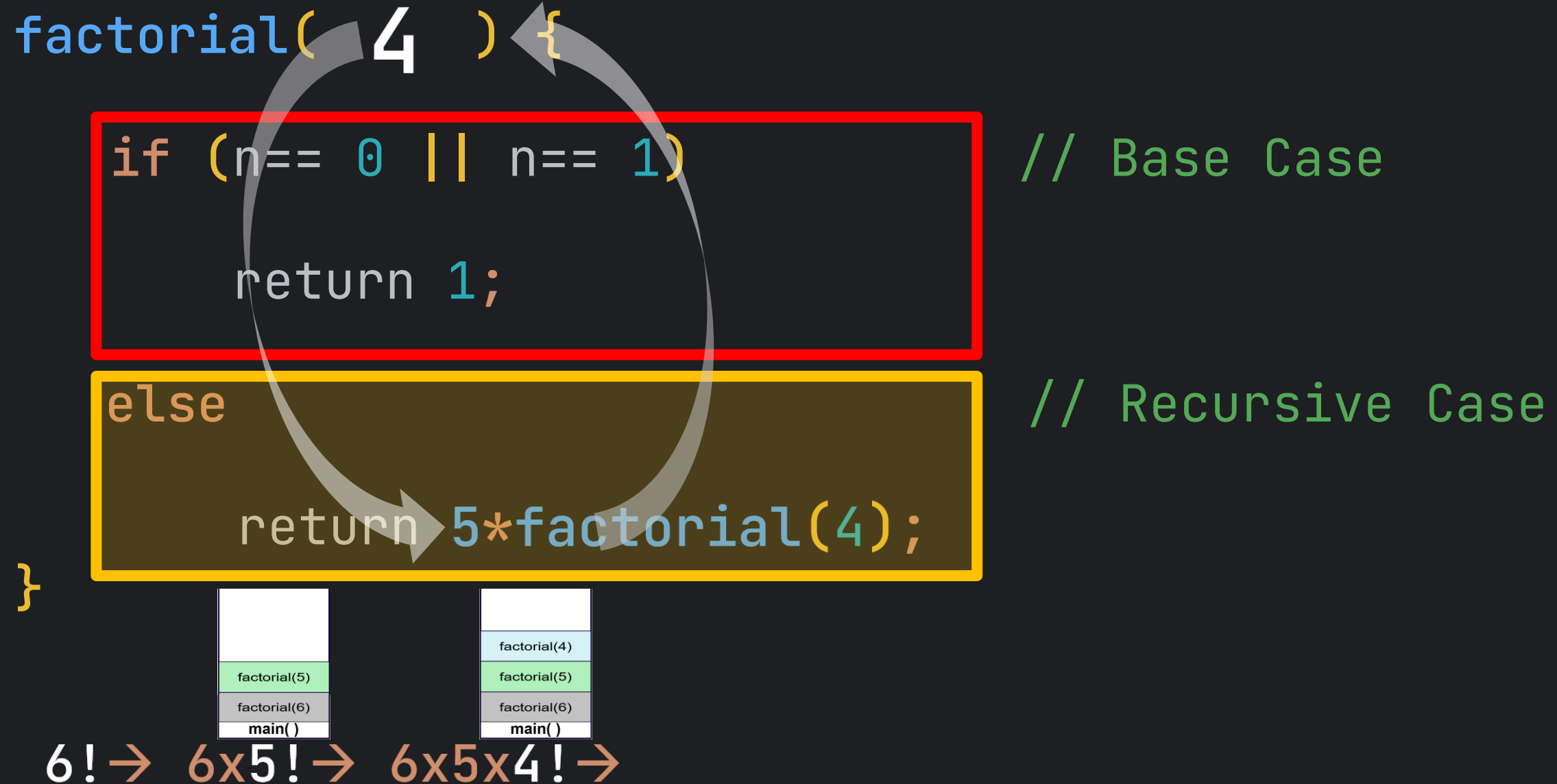
Recursion: a method calling itself

```
recursive(arg) {  
    if (baseCondition)           // Base Case  
        return baseValue  
    else                         // Recursive Case  
        return recursive(arg')  
}
```


Recursion: a method calling itself

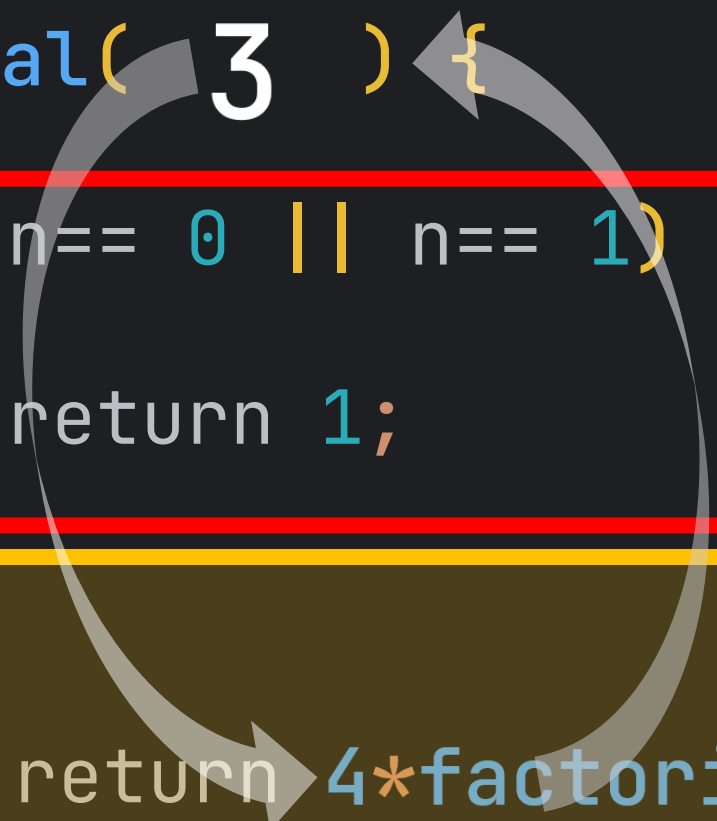


Recursion: a method calling itself



Recursion: a method calling itself

```
factorial( 3 ) {  
    if (n == 0 || n == 1) // Base Case  
        return 1;  
    else // Recursive Case  
        return 4*factorial(3);  
}
```



factorial(4)
factorial(5)
factorial(6)
main()

factorial(3)
factorial(4)
factorial(5)
factorial(6)
main()

6! → 6x5! → 6x5x4! →

Recursion: a method calling itself

```
factorial( 2 ) {
```

```
if (n == 0 || n == 1)
```

```
    return 1;
```

// Base Case

```
else
```

```
    return 3*factorial(2);
```

// Recursive Case

factorial(3)
factorial(4)
factorial(5)
factorial(6)
main()

Stack
factorial(2)
factorial(3)
factorial(4)
factorial(5)
factorial(6)
main()

6! → 6x5! → 6x5x4! → 6x5x4x3! →

Recursion: a method calling itself

```
factorial( 1 ) {
```

```
if (n == 0 || n == 1)
```

```
    return 1;
```

// Base Case

```
else
```

```
    return 2*factorial(1);
```

// Recursive Case

Stack
factorial(2)
factorial(3)
factorial(4)
factorial(5)
factorial(6)
main()

Stack
factorial(1)
factorial(2)
factorial(3)
factorial(4)
factorial(5)
factorial(6)
main()

$6! \rightarrow 6 \times 5! \rightarrow 6 \times 5 \times 4! \rightarrow 6 \times 5 \times 4 \times 3! \rightarrow 6 \times 5 \times 4 \times 3 \times 2! \rightarrow 6 \times 5 \times 4 \times 3 \times 2 \times 1!$

Recursion: a method calling itself

```
factorial( 1 ) {
```

```
if (n == 0 || n == 1)
```

// Base Case

```
return 1;
```

```
else
```

// Recursive Case

```
return 2*factorial(1);
```

Stack
factorial(1)
factorial(2)
factorial(3)
factorial(4)
factorial(5)
factorial(6)
main()

$6! \rightarrow 6 \times 5! \rightarrow 6 \times 5 \times 4! \rightarrow 6 \times 5 \times 4 \times 3! \rightarrow 6 \times 5 \times 4 \times 3 \times 2! \rightarrow 6 \times 5 \times 4 \times 3 \times 2 \times 1!$

Recursion: a method calling itself

```
factorial( 1 ) {
```

```
if (n== 0 || n== 1)
```

```
    return 1;
```

// Base Case

```
else
```

```
    return 2*factorial(1);
```

// Recursive Case

Stack
factorial(3)
factorial(4)
factorial(5)
factorial(6)
main()

$6! \rightarrow 6 \times 5! \rightarrow 6 \times 5 \times 4! \rightarrow 6 \times 5 \times 4 \times 3! \rightarrow 6 \times 5 \times 4 \times 3 \times 2$

Recursion: a method calling itself

```
factorial( 1 ) {
```

```
if (n == 0 || n == 1)
```

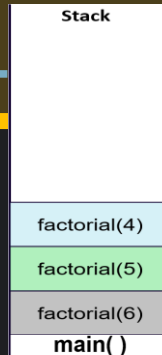
```
    return 1;
```

// Base Case

```
else
```

```
    return 2*factorial
```

// Recursive Case



$6! \rightarrow 6 \times 5! \rightarrow 6 \times 5 \times 4! \rightarrow 6 \times 5 \times 4 \times 3$

Recursion: a method calling itself

```
factorial( 1 ) {
```

```
if (n == 0 || n == 1)
```

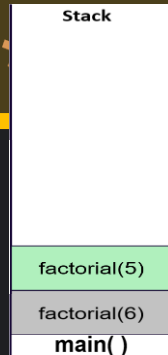
// Base Case

```
    return 1;
```

```
else
```

// Recursive Case

```
    return 2 * factorial(1);
```



6! → 6x5! → 6x5x24

Recursion: a method calling itself

```
factorial( 1 ) {
```

```
if (n== 0 || n== 1)
```

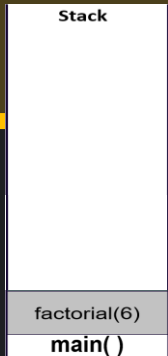
// Base Case

```
    return 1;
```

```
else
```

// Recursive Case

```
    return 2*factorial(1);
```



6! → 6x120

Recursion: a method calling itself

```
factorial( 1 ) {
```

```
if (n== 0 || n== 1)
```

// Base Case

```
    return 1;
```

```
else
```

// Recursive Case

```
    return 2*factorial(1);
```

Stack

main()

720



Fibonacci Sequence

The **Fibonacci series** is a sequence where each number is the **sum of the two preceding** ones:

- Starts with: 0, 1
- Formula:

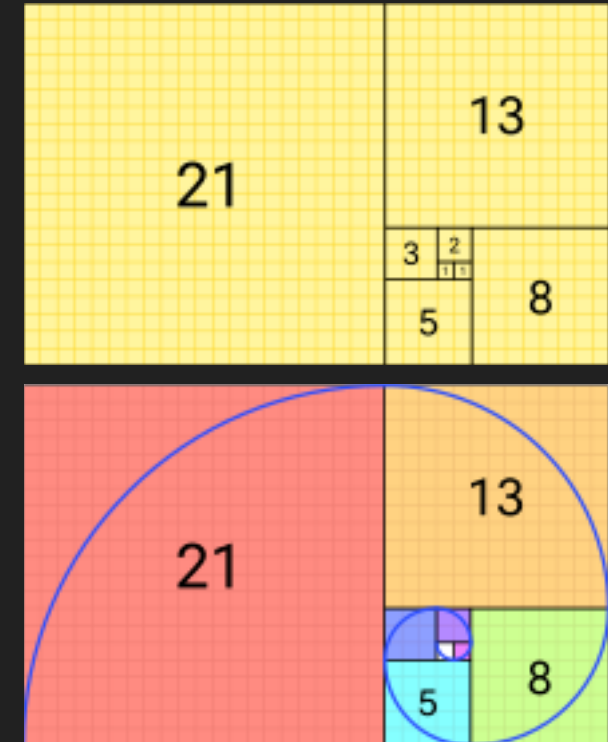
$$F(n) = F(n - 1) + F(n - 2), \quad \text{for } n \geq 2$$

Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89 ...

indices: 0 1 2 3 4 5 6 7 8 9 10 11 ...

$$f(0) = 0$$

$$f(1) = 1$$



Computefonacci